

Análisis Reto: GIRO
Sistema de Gestión de Inventario y Pedidos

Hansell Steven Benavides Jumbo - Sergi Montaña Galarza

28 de julio de 2026

Índice general

1. Capítulo 1: Introducción y Problemática	2
1.1. Introducción	2
1.2. Problemática	2
1.3. Metodología o estrategia de desarrollo	2
2. Capítulo 2: Planificación y análisis	3
2.1. Análisis de Actores y Modelo de Negocio	3
2.2. Requisitos funcionales	3
2.3. Requisitos no funcionales	4
2.4. Historias de Usuario / Casos de Uso	4
2.5. Sprint	7
2.6. Diagramas: Clases y BD (diseño lógico)	7
2.7. Diagrama de componentes	8
2.8. Arquitectura lógica y física (propuesta)	9
2.9. Prototipo de pantallas no funcional	10

Capítulo 1

Capítulo 1: Introducción y Problemática

1.1. Introducción

El presente documento tiene como propósito definir la visión general del Sistema de Gestión de Inventario y Pedidos B2B, denominado GIRO. Este sistema se establece en el contexto de la distribución mayorista, abarcando la atención de micromercados (tiendas pequeñas) en zonas tanto rurales como urbanas del Ecuador. Su objetivo principal es optimizar la vinculación comercial entre proveedores mayoristas y vendedores en ruta, reducir la pérdida de ventas mediante control de inventario en tiempo real y automatizar la toma de pedidos.

1.2. Problemática

La problemática central radica en la desconexión operativa entre el inventario real disponible de los proveedores mayoristas y la gestión de campo realizada por los vendedores en ruta al visitar micromercados. Los vendedores a menudo operan con información desactualizada, lo que genera pedidos sobre productos agotados, compromisos de entrega incumplidos y pérdidas financieras. Adicionalmente, la falta de un canal digital estructurado dificulta la gestión de solicitudes de vinculación comercial y el seguimiento del estado de los pedidos por parte de las tiendas pequeñas.

1.3. Metodología o estrategia de desarrollo

La estrategia de desarrollo se apoya en iteraciones ágiles, validación continua con stakeholders y la ejecución de *sprints* de refinamiento junto al equipo de desarrollo. Se incorporan también prácticas de análisis de negocio basadas en estándares del BABOK.

A nivel de arquitectura técnica, el proyecto utilizará **sqlite** como motor de base de datos relacional. Se seleccionó sqlite por su alta escalabilidad y la facilidad de implementación con el equipo actual. Para el desarrollo del backend, se empleará el framework **Django** (Python), el cual permite desarrollar de forma rápida, estructurada y segura la API REST que sirve de soporte tanto a la aplicación web (PWA) de los vendedores como a los paneles de gestión de los proveedores.

Capítulo 2

Capítulo 2: Planificación y análisis

2.1. Análisis de Actores y Modelo de Negocio

El análisis del sistema actual redefine el flujo operativo en torno a tres roles principales definidos en la arquitectura de datos:

- **Proveedor (Mayorista):** Dueño de los productos e inventario. Administra su catálogo, aprueba solicitudes de incorporación de vendedores y gestiona el ciclo de vida de los pedidos.
- **Vendedor:** Actor en ruta que explora proveedores verificados, envía solicitudes de asociación y registra pedidos en representación de las tiendas pequeñas, calculando sus comisiones.
- **Tienda Pequeña (Micromercado):** Negocio local que interactúa mediante solicitudes de visita o recepción de pedidos gestionados por su vendedor asignado.

2.2. Requisitos funcionales

A continuación, se listan los requisitos funcionales principales extraídos del análisis y adaptados a la arquitectura de la base de datos:

- **RF-01 (Gestión de Catálogo):** El sistema debe permitir al *Proveedor* registrar, actualizar y eliminar productos con control automático de SKU, stock disponible y precios.
- **RF-02 (Asociación de Vendedores):** El *Vendedor* debe poder explorar proveedores verificados, enviar una solicitud de vinculación con una descripción de intenciones y esperar la aprobación y asignación de comisión por parte del mayorista.
- **RF-03 (Toma de Pedidos en Ruta):** El *Vendedor* debe poder iniciar un pedido seleccionando una *Tienda*, agregando productos del catálogo del proveedor asociado y registrando el método de pago (Efectivo o Transferencia).
- **RF-04 (Seguimiento y Ciclo del Pedido):** El *Proveedor* debe poder avanzar el estado del pedido de forma secuencial (*'En proceso' → 'Enviado' → 'En ruta' → 'Entregado'*).

- **RF-05 (Solicitud de Visitas):** Las *Tiendas pequeñas* deben poder enviar solicitudes de visita a los vendedores de su sector para coordinar la reposición de inventario.

2.3. Requisitos no funcionales

Los requisitos de calidad técnica y restricciones operativas incluyen:

- **RNF-01 (Seguridad y Control de Acceso - RBAC):** El acceso a los diferentes módulos debe restringirse estrictamente mediante autenticación de Django y validación de perfiles (`perfil_proveedor`, `perfil_vendedor`), aplicando el principio de mínimo privilegio.
- **RNF-02 (Rendimiento):** La búsqueda y filtrado de productos en el catálogo del proveedor mediante consultas optimizadas (*icontains*) debe resolverse en el servidor en menos de 1.5 segundos.
- **RNF-03 (Seguridad en Tránsito):** Toda comunicación debe realizarse mediante HTTPS con TLS 1.3.
- **RNF-04 (Integridad de Datos):** Las operaciones críticas de inventario, creación de pedidos y aprobación de solicitudes deben ejecutarse bajo transacciones atómicas del ORM de Django para evitar inconsistencias relacionales.
- **RNF-05 (Disponibilidad):** Garantizar un *uptime* mínimo del 99.5 % mensual.

2.4. Historias de Usuario / Casos de Uso

En esta sección se modela la interacción entre los actores del sistema GIRO y las funcionalidades principales mediante diagramas de casos de uso orientados a los tres roles reales de la plataforma.

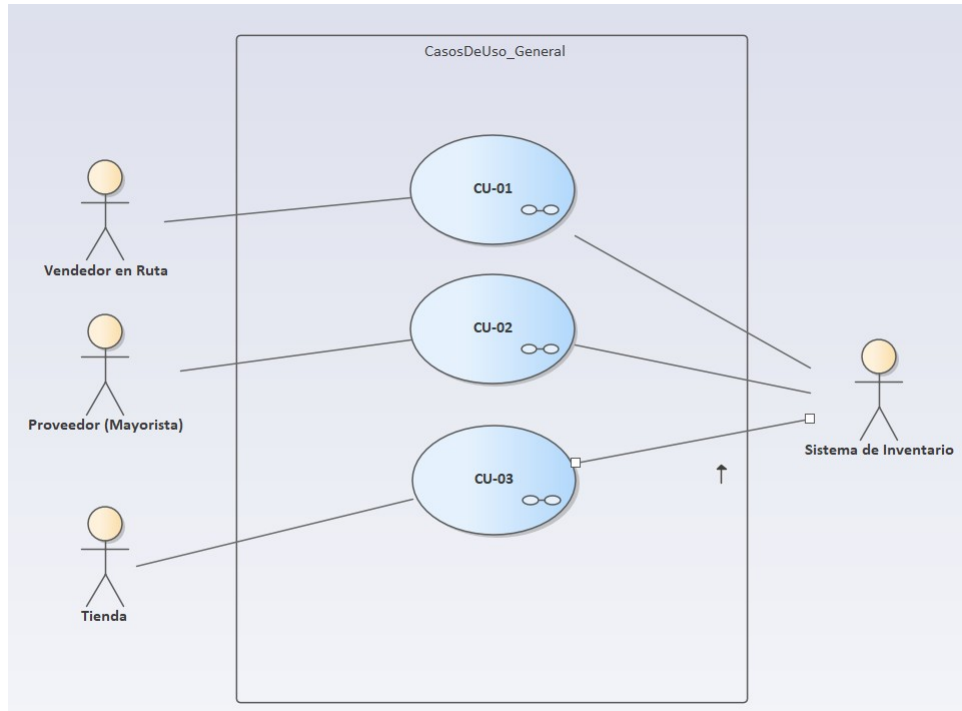


Figura 2.1: Diagrama de Casos de Uso General

En la Figura 2.1 se ilustra la visión global del sistema. Se identifican los actores actualizados (Proveedor, Vendedor y Tienda Pequeña) interactuando con los módulos centrales de gestión y transaccionalidad.

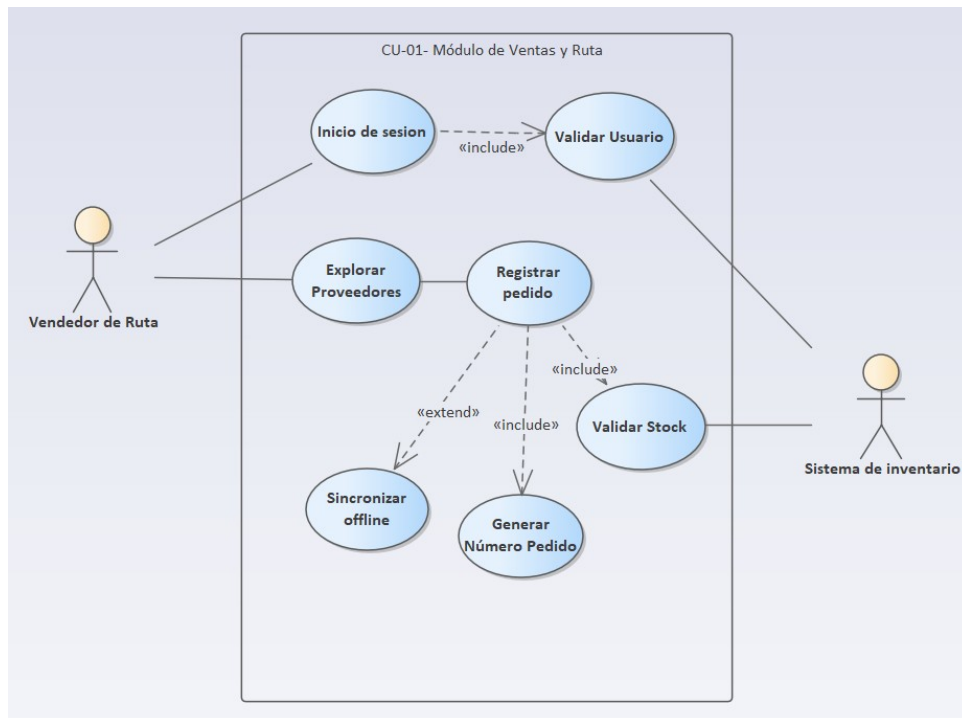


Figura 2.2: Flujo Principal: Registro de pedido en campo

La Figura 2.2 detalla el Caso de Uso de toma de pedidos. Aquí se observa cómo el

Vendedor inicia sesión con su perfil validado y procede a procesar el pedido vinculado a la tienda correspondiente.

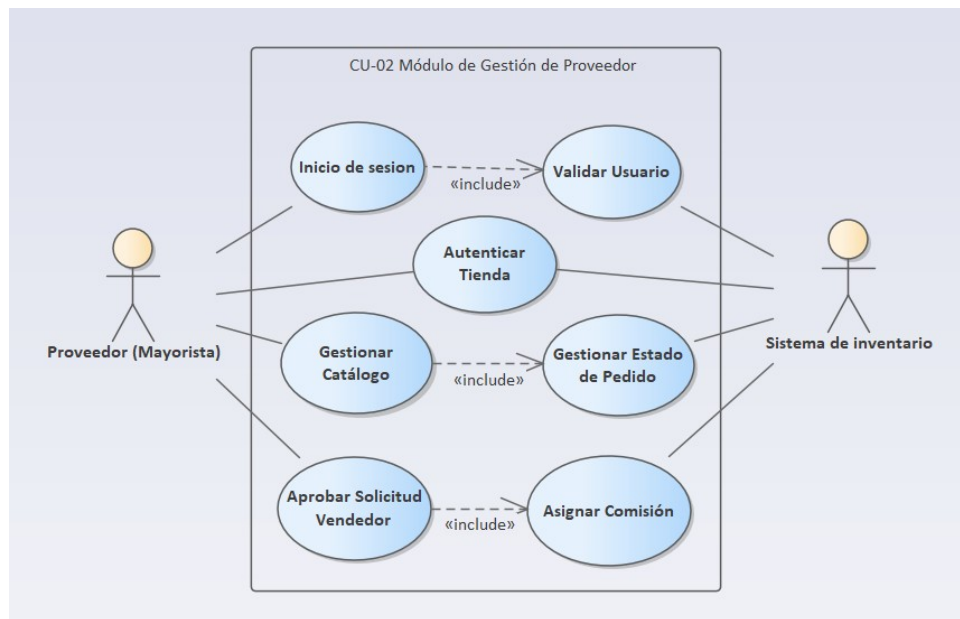


Figura 2.3: Validaciones del Registro de Pedido

Como se detalla en la Figura 2.3, el registro del pedido tiene dependencias estrictas (*includes*): requiere validar el stock disponible en el catálogo del proveedor y verificar la existencia de una relación comercial activa.

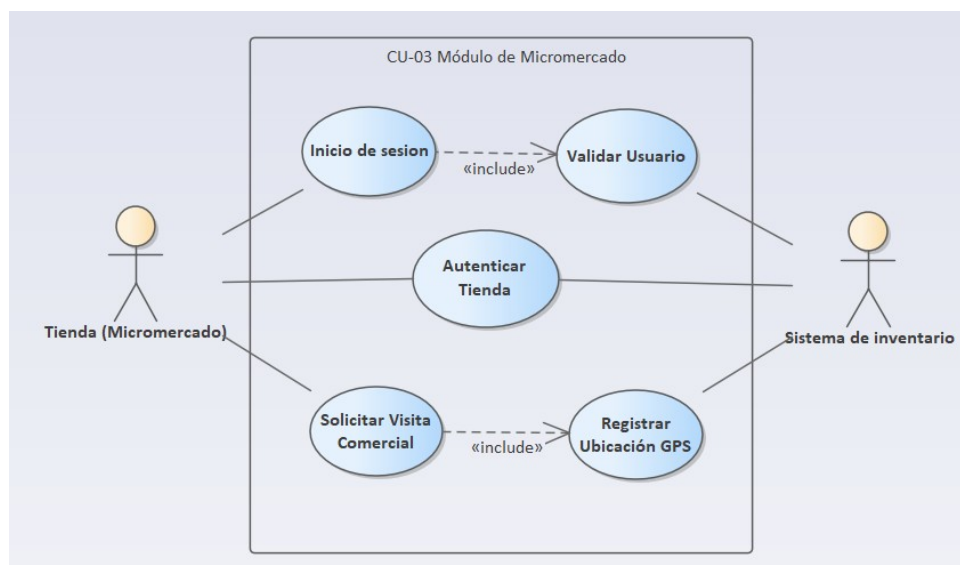


Figura 2.4: Extensión Offline del Registro de Pedido

Finalmente, la Figura 2.4 resalta la funcionalidad *extend* del sistema: en caso de que el vendedor se encuentre en una zona rural sin cobertura, el sistema extiende su comportamiento para habilitar el almacenamiento local cifrado, sincronizando posteriormente los datos al recuperar la señal.

2.5. Sprint

La estrategia de desarrollo se divide en iteraciones o *Sprints* enfocados en entregar valor incremental alineado a los módulos críticos del sistema:

- **Sprint 1: Base, Autenticación y Catálogo.** Configuración de la base de datos MariaDB, despliegue del backend en Django, control de acceso basado en roles (RBAC) y la gestión de productos por parte del Proveedor.
- **Sprint 2: Módulo Web de Campo y Venta.** Desarrollo de la interfaz para vendedores, flujo de exploración de proveedores, envío de solicitudes de vinculación y el registro operativo de pedidos para las tiendas.
- **Sprint 3: Supervisión, Estados y Logística.** Implementación de la gestión de estados de pedidos por parte del mayorista (*En proceso*, *Enviado*, *En ruta*, *Entregado*) y el submódulo de solicitudes de visita.

2.6. Diagramas: Clases y BD (diseño lógico)

A continuación, se presenta el diseño estructural del sistema, alineado de forma estricta con la implementación real del código backend (`models.py`).

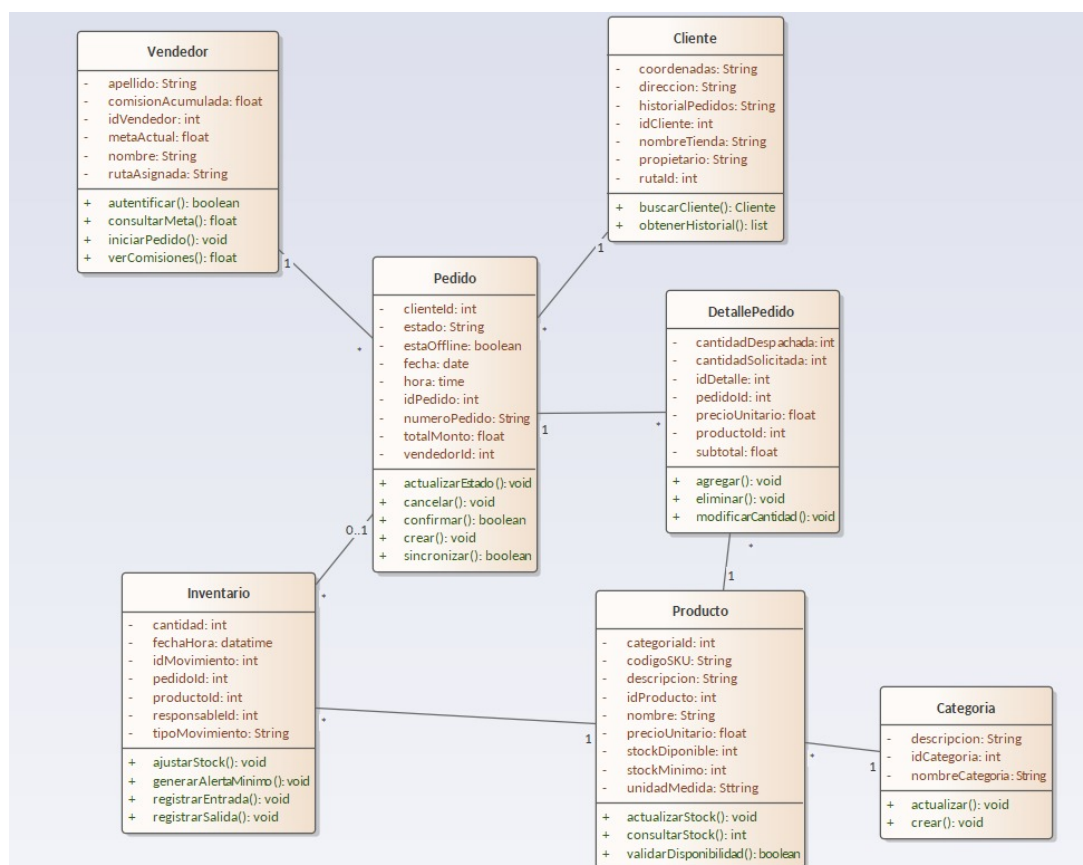


Figura 2.5: Diagrama de Clases (Lógica de Negocio)

La Figura 2.5 muestra el diagrama de clases de UML actualizado. Este modelo define las entidades principales del dominio: **User**, **Proveedor**, **Vendedor**, **Tienda**, **SolicitudVendedor**,

Producto, Categoria, Pedido, DetallePedido y SolicitudVisita, detallando sus atributos y métodos de operación conforme al ORM.

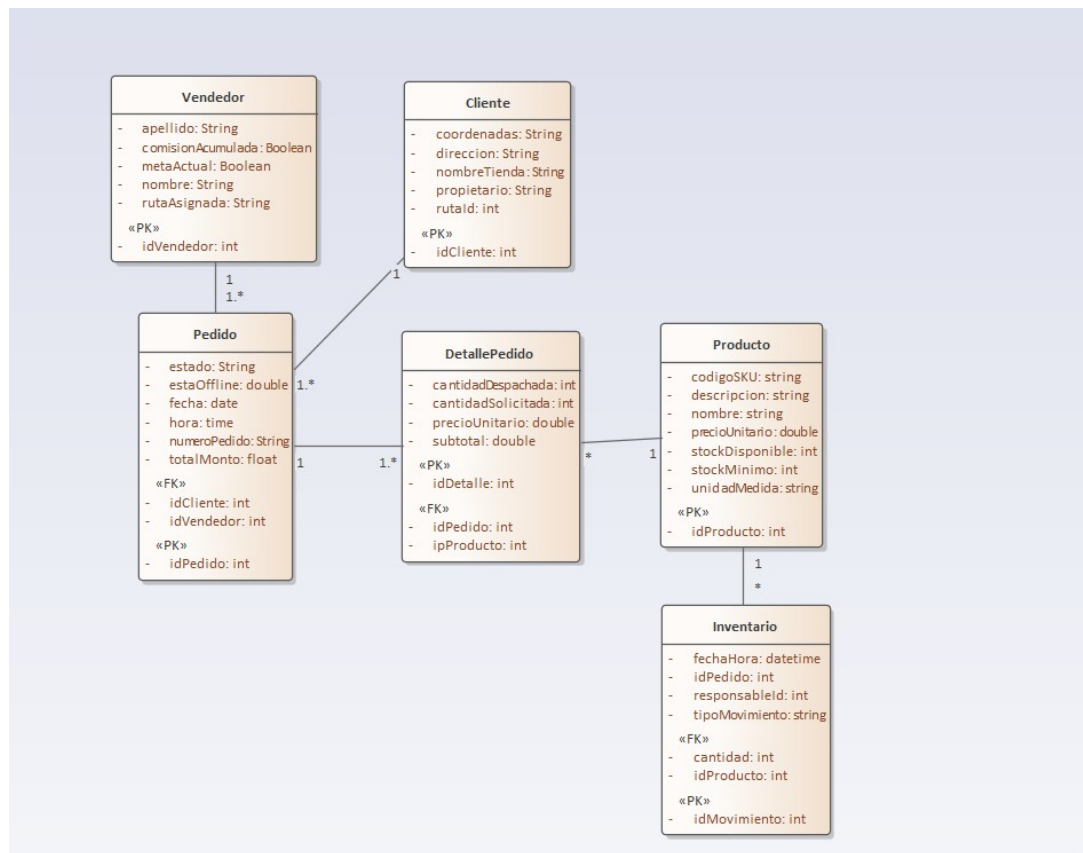


Figura 2.6: Diagrama Lógico de Base de Datos para Django

La Figura 2.6 representa la estructura física adaptada para Django. En este diagrama se visualizan las Llaves Primarias («PK») y Llaves Foráneas («FK») que garantizan la integridad referencial entre los perfiles de usuario, catálogos de proveedores y las transacciones de pedidos.

2.7. Diagrama de componentes

El sistema distribuye sus funcionalidades en tres capas principales que agrupan los componentes de software, garantizando el aislamiento de responsabilidades y la escalabilidad del proyecto.

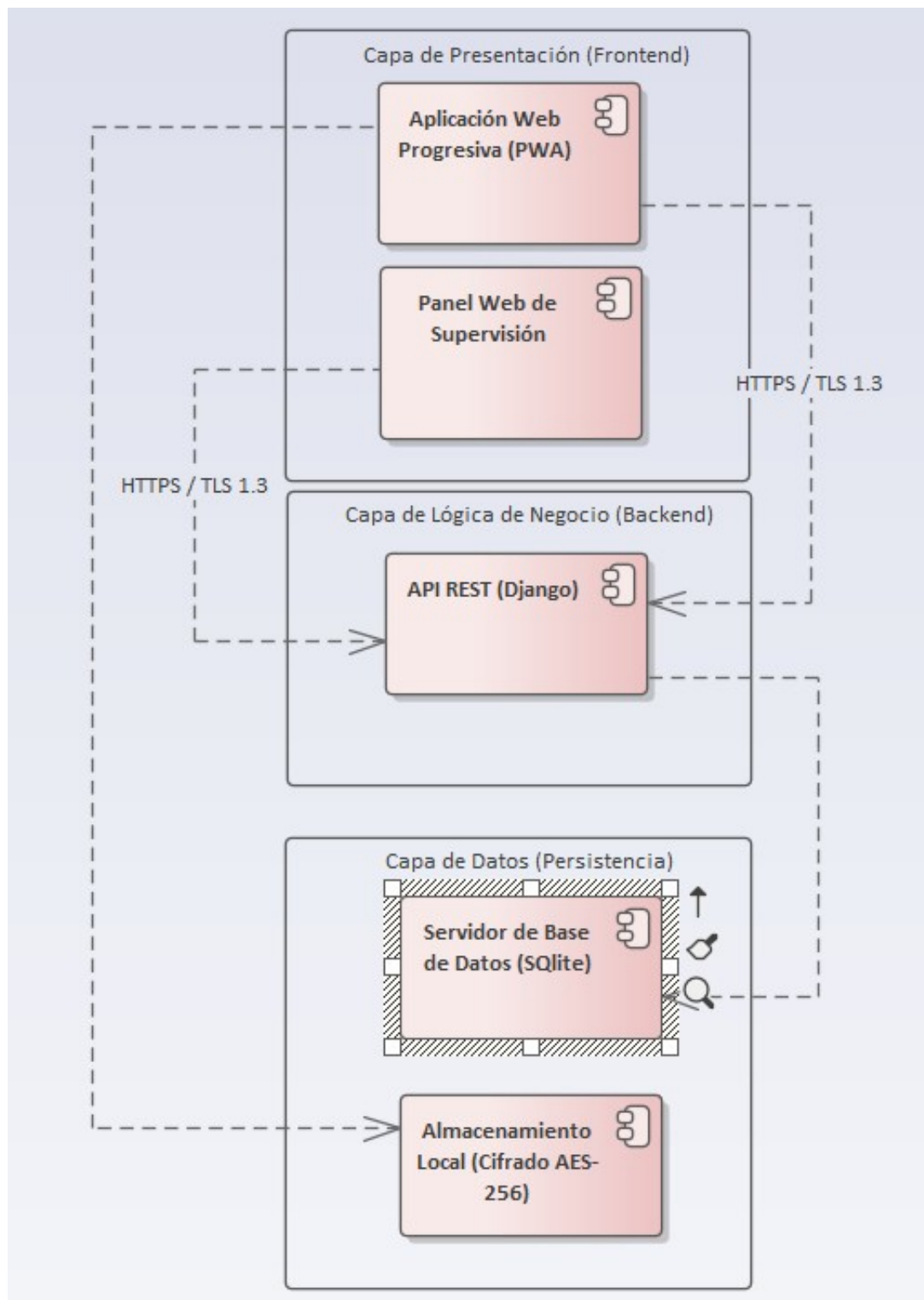


Figura 2.7: Diagrama de Componentes (Arquitectura en 3 Capas)

En la Figura 2.7 se ilustra la arquitectura orientada a servicios del sistema GIRO. La **Capa de Presentación** contiene la interfaz web responsiva para los vendedores en ruta y los paneles de control web para los proveedores mayoristas. Estos componentes se comunican de forma segura (mediante HTTPS / TLS 1.3) con la **Capa de Lógica de Negocio**, donde la API desarrollada en Django procesa las peticiones, validaciones de perfiles y reglas de negocio. Finalmente, la **Capa de Datos** garantiza la persistencia

centralizada mediante el servidor MariaDB.

2.8. Arquitectura lógica y física (propuesta)

El sistema GIRO adopta una arquitectura orientada a servicios, diseñada para alta disponibilidad y el manejo de contextos con baja conectividad:

- **Arquitectura Lógica:** El frontend opera como una aplicación web con capacidades *offline-first*, sincronizando datos asincrónicamente con el backend. Este último expone servicios que centralizan la lógica de validación de stock y el control de solicitudes comerciales.
- **Arquitectura Física:** Los usuarios acceden mediante navegadores web desde smartphones o computadoras de escritorio. El servidor web y la base de datos se alojan en una infraestructura en la nube que permite el escalamiento horizontal, garantizando un *uptime* del 99.5 %.

2.9. Prototipo de pantallas no funcional

Las interfaces web están diseñadas con un enfoque responsivo para adaptarse a pantallas móviles, cumpliendo con la restricción de diseño de botones táctiles amplios para el uso ágil en campo. A continuación, se presenta el flujo principal para la toma de pedidos.

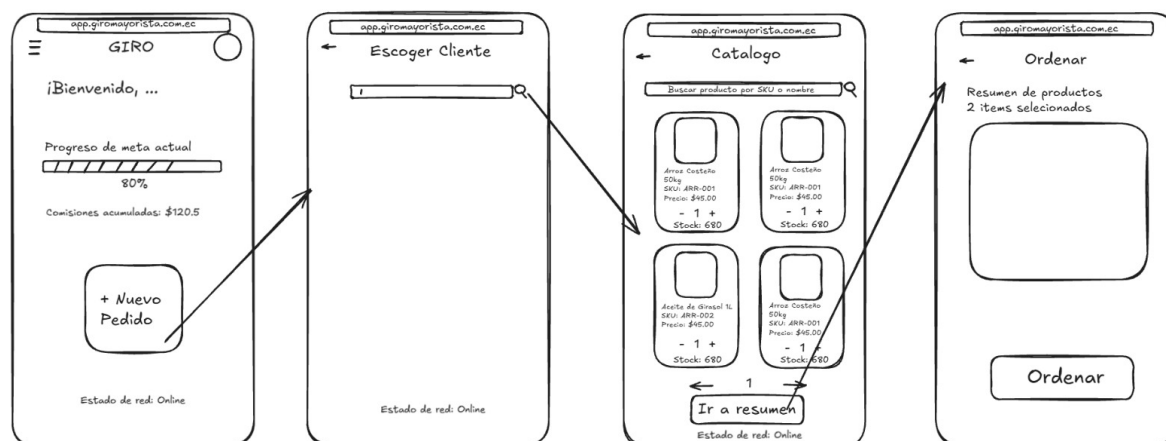


Figura 2.8: Wireframe del Flujo de Venta en Ruta (PWA)

La Figura 2.8 detalla el prototipo de la secuencia de pantallas que utiliza el vendedor durante su ruta:

- **Dashboard (Inicio):** Presenta un resumen del rendimiento del vendedor (progreso de meta actual y comisiones acumuladas) junto con accesos directos para la gestión de pedidos.
- **Escoger Cliente:** Una interfaz limpia equipada con una barra de búsqueda que permite filtrar y seleccionar rápidamente el micromercado a gestionar.

- **Catálogo:** Despliega los productos disponibles del proveedor asociado en formato de tarjetas, indicando su SKU, precio y *stock* actualizado. Incluye selectores numéricos de cantidades y un botón para avanzar al resumen.
- **Ordenar (Resumen):** Pantalla final que muestra la consolidación de los ítems seleccionados y el método de pago antes de confirmar el envío hacia el sistema del proveedor mayorista.